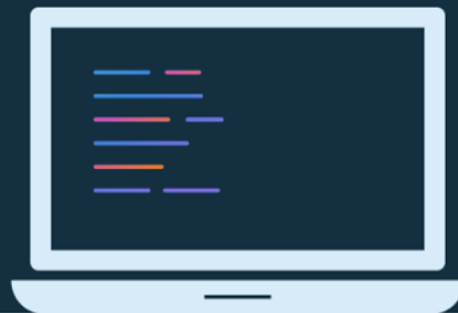




Android Development with Kotlin



List View

ListView

- ListView is a view group that displays a list of scrollable items.
- The list items are automatically inserted into the list using an **Adapter** that pulls content from a source such as:
 - an array or
 - database query

and converts each item result into a view that's placed into the list.

ListAdapter

That is the bridge between a ListView and the data to fill the list.

ListAdapter (known Indirect Subclasses)

ArrayAdapter

- used if the model data is an array of objects

BaseAdapter

- used if the model data is some other in-memory data structure

SimpleAdapter

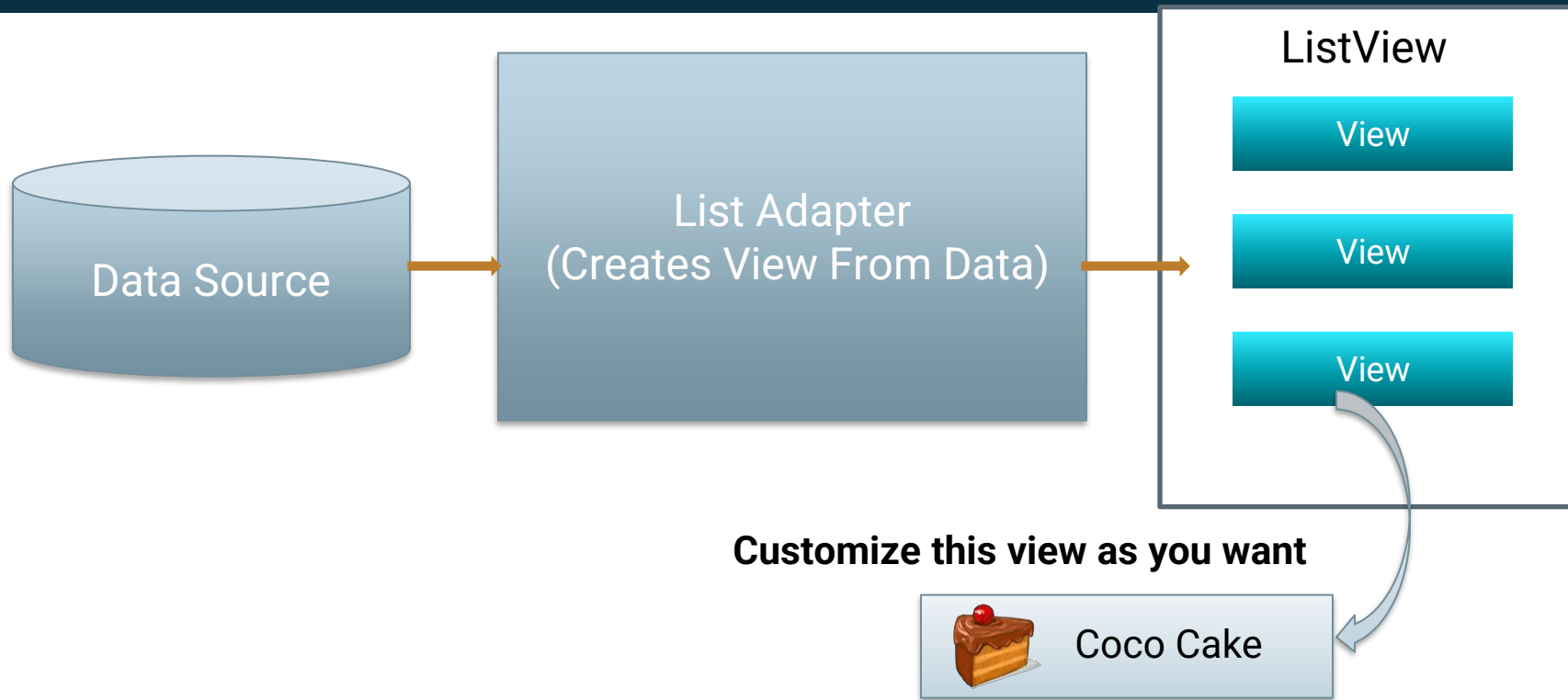
- used for static data.

HeaderViewListAdapter

- used when a ListView has header views.



ListView cont'd



Create Simple ListView

1. Define Array or Data Source

```
val values: ArrayList<String> = arrayListOf("Saturday", "Sunday")
```

2. Tell the List Adapter how to display the data

```
val adapter: ArrayAdapter<String> =  
    ArrayAdapter(applicationContext,  
                 android.R.layout.simple_list_item_1, values)  
listView.adapter = adapter
```

Create Simple ListView cont'd

3. Define what happens when a user interacts with a row

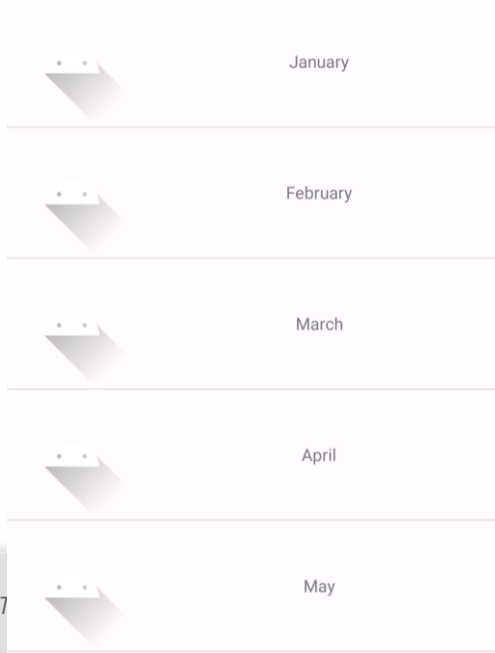
```
listview.onItemClickListener =  
    OnItemClickListener { parent, view, position, id ->  
        Toast.makeText(  
            applicationContext,  
            "You clicked ${parent?.getItemAtPosition(position)}",  
            Toast.LENGTH_LONG  
        ).show()  
    }
```


Custom List View

Custom ListView

By default, ArrayAdapter assumes that a single row is represented by a TextView, but we can customize this view

Display static image icon in ListView



Custom ListView cont'd

1. Define Array or Data Source

```
val values: ArrayList<String> = arrayListOf("January", "February")
```

Custom ListView cont'd

2. Define single-row appearance in a separate layout

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@drawable/ic_launcher_foreground" />

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:layout_gravity="center"
        android:text="TextView" />
</LinearLayout>
```

single_row.xml

Custom ListView cont'd

3. Tell the List Adapter how to display the data

```
val adapter: ArrayAdapter<String> =  
    ArrayAdapter(applicationContext,  
        R.layout.single_row,  
        R.id.textView,  
        values)  
listView.adapter = adapter
```



Layout Folder
Text View Id

Custom ListView cont'd

- **How Display Complex ListView?**
 1. Define Array or Data Source
 2. Define single-row appearance in a separate layout
 3. Tell the List Adapter how to display the data by customizing this adapter



Custom ListView cont'd

- **How to create a custom adapter?**
 1. Create a class that extends ArrayAdapter.
 2. Define a constructor that calls one of the predefined superclass constructors.
 3. Override getView() method.

Layout Inflater

- **What is Layout Inflation?**

Converting XML appearance definition into View objects in code and set the values for all attributes

- **How does it work?**

```
val inflater: LayoutInflater =  
    context.getSystemService(Context.LAYOUT_INFLATER_SERVICE) as LayoutInflater  
val row: View = inflater.inflate(R.layout.custom_listview, parent, false)
```


Custom ListView cont'd

```
class SimpleArrayAdapter(context: Context, var values: Array<String>, var descriptions: Array<String>,
    var images: IntArray) : ArrayAdapter<Any?>(context, R.layout.custom_listview, R.id.text_view_name,
        values) {
    override fun getView(position: Int, convertView: View?, parent: ViewGroup) : View {
        // Inflating the layout
        val inflater: LayoutInflater =
            context.getSystemService(Context.LAYOUT_INFLATER_SERVICE) as LayoutInflater
        val row: View = inflater.inflate(R.layout.custom_listview, parent, false)

        // Get the reference to the view object
        val imageView = row.findViewById<ImageView>(R.id.imageView)
        val name = row.findViewById<TextView>(R.id.text_view_day)
        val description = row.findViewById<TextView>(R.id.text_view_description)

        // Providing the element of an array by specifying its position
        imageView.setImageResource(images[position])
        name.text = values[position];           description.text = descriptions[position];
        return row
    }
}
```



Custom ListView cont'd

There is a problem

`java.lang.OutOfMemoryError`

We have a limited heap size

Custom ListView cont'd

- Images come in all shapes and sizes.
- In many cases they are larger than required for a typical application user interface (UI).
- For example, the system Gallery application displays photos taken using your Android device's camera which are typically much higher resolution than the screen density of your device.

Custom ListView cont'd

- **Solution**

Loading Large Bitmaps
Efficiently with **decoding**

```
fun decodeSampleBitmapFromResource(
    res: Resources,
    resId: Int,
    reqWidth: Int,
    reqHeight: Int
): Bitmap {
    // First decode with inJustDecodeBounds=true to check dimensions
    val options = BitmapFactory.Options()
    options.inJustDecodeBounds = true
    BitmapFactory.decodeResource(res, resId, options)

    // Calculate inSampleSize
    options.inSampleSize = calculateInSample(options, reqHeight, reqWidth)

    // Decode bitmap with inSampleSize set
    options.inJustDecodeBounds = false

    return BitmapFactory.decodeResource(res, resId, options)
}
```



Custom ListView cont'd

```
fun calculateInSample(options: BitmapFactory.Options, reqHeight: Int, reqWidth: Int): Int {  
    // Raw height and width of image  
    val height : Int = options.outHeight  
    val width : Int = options.outWidth  
    var inSampleSize = 1  
  
    if (height > reqHeight || width > reqWidth) {  
        val halfHeight : Int = height / 2  
        val halfWidth : Int = width / 2  
  
        // Calculate the largest inSampleSize value that is a power of 2 and  
        // keeps both height and width larger than the requested height and width  
        while ((halfHeight / inSampleSize) > reqHeight && (halfWidth / inSampleSize) > reqWidth) {  
            inSampleSize *= 2  
        }  
    }  
  
    return inSampleSize  
}
```



Custom ListView cont'd

There is a problem

What if the records increased

Custom ListView cont'd

Solution (Best Practice)

using ViewHolder to reuse the unused views

Custom ListView cont'd

```
class ViewHolder(var convertView: View) {  
    private var day: TextView? = null  
    private var description: TextView? = null  
    private var imageView: ImageView? = null  
  
    fun getDay(): TextView? {  
        return day ?: convertView.findViewById(R.id.text_view_day)  
    }  
  
    fun getDescription(): TextView? {  
        return description ?: convertView.findViewById(R.id.text_view_description)  
    }  
  
    fun getImageView(): ImageView? {  
        return imageView ?: convertView.findViewById(R.id.imageView)  
    }  
}
```



```

class MyAdapter(
    private var context: Context,
    var days: Array<String>,
    var descriptions: Array<String>,
    var image: IntArray
) : ArrayAdapter<Any?>(context, R.layout.custom_listview, R.id.text_view_day, days) {

    override fun getView(position: Int, convertView: View?, parent: ViewGroup): View {
        var rowView : View? = convertView
        val viewHolder: ViewHolder
        if (rowView == null) {
            // Inflating the layout
            val inflater : LayoutInflater =
                context.getSystemService(Context.LAYOUT_INFLATER_SERVICE) as LayoutInflater
            rowView = inflater.inflate(R.layout.custom_listview, parent, attachToRoot: false)

            viewHolder = ViewHolder(rowView)
            rowView.tag = viewHolder
            Log.i( tag: "CUSTOM", msg: "first time")
        } else {
            viewHolder = rowView.tag as ViewHolder
            Log.i( tag: "CUSTOM", msg: "second time")
        }

        // Providing the element of an array by specifying its position
        viewHolder.getDay()?.text = days[position]
        viewHolder.getDescription()?.text = descriptions[position]
        viewHolder.getImageView()?.setImageResource(image[position])
        return rowView!!
    }
}

```

Displaying lists with RecyclerView

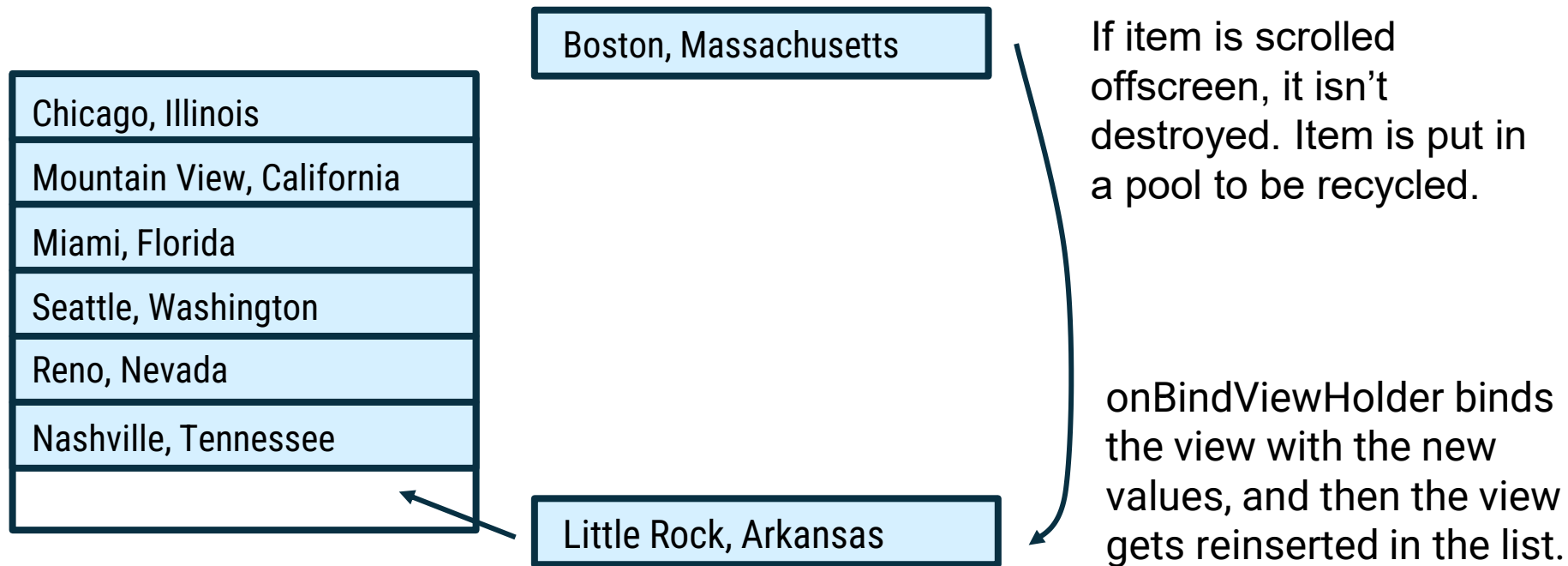
RecyclerView

- Widget for displaying lists of data
- "Recycles" (reuses) item views to make scrolling more performant
- Can specify a list item layout for each item in the dataset
- Supports animations and transitions

RecyclerView.Adapter

- Supplies data and layouts that the RecyclerView displays
- A custom Adapter extends from `RecyclerView.Adapter` and overrides these three functions:
 - `getItemCount`
 - `onCreateViewHolder`
 - `onBindViewHolder`

View recycling in RecyclerView



RecyclerView in Steps

1. Define a model class to use as the data source
2. Add a RecyclerView to your layout to display the items
3. Create a custom row layout XML file to visualize the item
4. Create a RecyclerView.Adapter and ViewHolder to render the item
5. Bind the adapter to the data source to populate the RecyclerView

Add RecyclerView to your layout

```
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/rv"  
    android:scrollbars="vertical"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"/>
```

Create a list item layout

```
res/layout/item_view.xml
```

```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <TextView
        android:id="@+id/number"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</FrameLayout>
```


Create a list adapter

```
class MyAdapter(val data: List<Int>) : RecyclerView.Adapter<MyAdapter.MyViewHolder>() {  
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {  
        val layout = LayoutInflater.from(parent.context).inflate(R.layout.item_view,  
            parent, false)  
        return MyViewHolder(layout)  
    }  
    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {  
        holder.textView.text = data.get(position).toString()  
    }  
    override fun getItemCount(): Int = data.size  
  
    class MyViewHolder(val row: View) : RecyclerView.ViewHolder(row) {  
        val textView = row.findViewById<TextView>(R.id.number)  
    }  
}
```

Set the adapter on the RecyclerView

In MainActivity.kt:

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
  
    val rv: RecyclerView = findViewById(R.id.rv)  
    rv.layoutManager = LinearLayoutManager(this)  
  
    rv.adapter = MyAdapter(IntRange(0, 100).toList())  
}
```

Demo

Assignment 1

Create an Activity that has a ListView display the day of the weeks

- When clicking on any row, displays the name of the day on a Toast

Saturday

Sunday

Monday

Tuesday

Wednesday

Thursday

Friday

Assignment 2

Create an application that displays a recycler view with each row consisting of:

- ImageView
- 2 TextView